Rustlings

- Site: https://rustlings.rust-lang.org
- Repo: https://github.com/rust-lang/rustlings
- Note: Rustlings isn't really a "book", but I needed someplace to make notes and save examples. In theory I could just re-init it and re-work any of the exercises, but I like having a place to keep reference to stuff I find personally useful rather than having to start over with a resource.

Running Rustlings

From this folder (`rust/rustlings`), start the interactive watcher:

```
rustlings
```

- Watch mode monitors files in `exercises/` , re-compiles on save, runs tests, and advances when green (state stored in `.rustlings-state.txt`).
- Interactive hotkeys:
  - `h` : Show hint for the current exercise.
  - `l` : List all exercises and their status.
  - `c` : Re-run/check the current exercise.
  - `q` : Quit watch mode.
- One-off CLI commands:
  - `rustlings run <name>` : Run a single exercise (e.g. `rustlings run vecs1`).
  - `rustlings hint <name>` : Print hint for a specific exercise.
  - `rustlings reset <name>` : Reset an exercise back to initial state.

Plan

Work through it alongside the Brown version of The Rust Programming Language (TRPL). Do the exercises in order, pausing at the first one whose chapter I haven't read yet:

Exercises	Read first	Notes
`01` – `04`	TRPL 3	
`05` – `11`	TRPL 4–8	`05_vecs` needs ch8; `08_enums` needs §19.3
`12` – `16`	TRPL 9–10	`12_options` needs ch10
`17_tests`	TRPL 11	
`18_iterators`	TRPL 12–13	
`19` – `20`	TRPL 14–16	
`21_macros`	TRPL 19–20	
`22` – `23`	finish TRPL	`22_clippy` is Appendix D
`24_async`	TRPL 17	

Exercises and TRPL Chapters

Exercise	TRPL Chapter
`01_variables`	§3.1
`02_functions`	§3.3
`03_if`	§3.5
`04_primitive_types`	§3.2, §4.3
`05_vecs`	§8.1
`06_move_semantics`	§4.1-2
`07_structs`	§5.1, §5.3
`08_enums`	§6, §19.3
`09_strings`	§8.2
`10_modules`	§7
`11_hashmaps`	§8.3

Exercise	TRPL Chapter
`12_options`	§10.1
`13_error_handling`	§9
`14_generics`	§10
`15_traits`	§10.2
`16_lifetimes`	§10.3
`17_tests`	§11.1
`18_iterators`	§13.2-4
`19_smart_pointers`	§15, §16.3
`20_threads`	§16.1-3
`21_macros`	§20.5
`22_clippy`	Appendix D
`23_conversions`	n/a
`24_async`	§17

Notes

- Overall, I liked Rustlings. It is a much-needed pairing to TRPL given how theory-heavy and practice-light that book is. If you are working from another book that already has a hands-on component, it is largely unnecessary since most exercises will feel trivial compared to building real projects.
- The conversion section was a highlight because something really needs to introduce `From` and `Into` to beginners (a notable gap in TRPL), though *100 Exercises to Learn Rust* also covers that ground.
- Beyond that, it is a relatively quick set of exercises to blow through. Seeing the compiler output on the few exercises that were not immediately obvious provided good diagnostic and debugging practice.
- In terms of value, it is about on par with the Brown interactive TRPL in- chapter comprehension quizzes (not the more challenging ownership and design quizzes): it

verifies you were paying attention while reading, but does not offer much beyond that.

- Most exercises were trivial. I forgot there was an easy way to initialize an array to 100 elements of the same value and looked up a complicated solution to converting a range to an array, but then was reminded after the fact that I can just do `let a = [0; 100]`. The solutions had the correct way to do it.
- It's a nice touch that the solutions are all empty until after you solve the problem. I have only had to use the hints once or twice, but they seem thorough enough to get you going when your stuck.

18 - Iterators

- The `collect` method is generic over `FromIterator`, meaning the destination container is inferred purely from type annotations or function return types.

String vs Vector Collection (`iterators2.rs`)

- The exact same pipeline produces a `Vec<String>` or concatenates directly into a single `String` with zero changes to the function body:

```
fn capitalize_words_vector(words: &[&str]) -> Vec<String> {
    words.iter().map(|s| capitalize_first(s)).collect()
}

fn capitalize_words_string(words: &[&str]) -> String {
    words.iter().map(|s| capitalize_first(s)).collect()
}
```

Transposing `Result` Collections (`iterators3.rs`)

- An iterator yielding `Result<T, E>` can be collected into `Vec<Result<T, E>>` (preserving each outcome) or inverted into `Result<Vec<T>, E>` (short-circuiting on the first `Err`):

```rust
// Inverts into Result<Vec<T>, E> (short-circuits on first error):
fn result_with_list() -> Result<Vec<i64>, DivisionError> {
    let numbers = [27, 297, 38502, 81];
    numbers.into_iter().map(|n| divide(n, 27)).collect()
}

// Retains every individual outcome as Vec<Result<T, E>>:
fn list_of_results() -> Vec<Result<i64, DivisionError>> {
    let numbers = [27, 297, 38502, 81];
    numbers.into_iter().map(|n| divide(n, 27)).collect()
}
```

Product Adapter for Factorial (`iterators4.rs`)

- Range `.product()` avoids recursion, loops, and mutable state in one line:

```rust
fn factorial(num: u64) -> u64 {
    (2..=num).product()
}
```

Filtering and Summing Collections (`iterators5.rs`)

- Reusing helper functions in loops vs composing with `.map(...).sum()` :

```rust
fn count_iterator(map: &HashMap<String, Progress>, value: Progress) -> usize
{
    map.values().filter(|v| **v == value).count()
}

fn count_collection_for(
    collection: &[HashMap<String, Progress>],
    value: Progress,
) -> usize {
    let mut count = 0;
    for map in collection {
        count += count_for(map, value);
    }
    count
}

fn count_collection_iterator(
    collection: &[HashMap<String, Progress>],
    value: Progress,
) -> usize {
    collection.iter().map(|m| count_iterator(m, value)).sum()
}
```

19 - Smart Pointers (`Cow`)

- `Cow` stands for Clone-On-Write and is defined in `std::borrow::Cow`. Under the hood, it is simply an enum with two variants: `Borrowed(&'a B)` and `Owned(<B as ToOwned>::Owned)`.
- Deref coercion makes reads transparent: operations like `input.len()` and indexing `input[ind]` automatically dereference to the underlying slice (`[i32]`) with zero cost, regardless of whether it is borrowed or owned.
- Mutation requires `input.to_mut()`, which lazily clones borrowed data into an owned container only when a write actually occurs.
- Testing variant transitions with `matches!` : `matches!(input, Cow::Owned(_))` is shorthand for a `match` expression testing if `input` is currently the `Owned` variant (with `_` matching any inner value).
- Lifecycle summary:

Starts As	Mutated?	Resulting Variant	Allocation / Clone?
`Cow::Borrowed`	No	`Cow::Borrowed`	No (zero-cost borrow)

Starts As	Mutated?	Resulting Variant	Allocation / Clone?
`Cow::Borrowed`	Yes	`Cow::Owned`	Yes (cloned on write)
`Cow::Owned`	No	`Cow::Owned`	No
`Cow::Owned`	Yes	`Cow::Owned`	No (mutated in place)

20 - Threads (Joining Handles Without a Loop)

- There is no `join_all` in `std::thread` (that is from async runtimes like `futures` / Tokio). In standard library code, joining a collection of thread handles is often written with a loop:

```
let mut results = Vec::new();
for handle in handles {
    results.push(handle.join().unwrap());
}
```

- Instead of a manual loop, you can use iterator adapters in two ways:

```
// 1. Direct unwrap inside map:
let results: Vec<_> = handles
    .into_iter()
    .map(|h| h.join().unwrap())
    .collect();

// 2. Invert Vec<Result<T, E>> into Result<Vec<T>, E>:
let results: Vec<_> = handles
    .into_iter()
    .map(|h| h.join())
    .collect::<Result<Vec<_>, _>>()
    .unwrap();
```

- Rule of thumb for `map(...).unwrap()` vs `collect::<Result<...>>()`:
  - `.map(|x| f(x).unwrap()).collect()`: Good for quick scripts, tests, or prototypes where any failure should immediately panic and crash.
  - `.map(f).collect::<Result<Vec<_>, _>>()?`: Production-grade Rust where errors should be handled, logged, or propagated up the call stack with `?` instead of crashing the thread.

21 - Macros (Scoping and Syntax)

- Definition order and scoping:
  - `macro_rules!` definitions are evaluated strictly top-to-bottom textually. Unlike functions, a macro cannot be invoked before its definition appears in the source file.
  - To use a macro defined inside a submodule, you can annotate the macro with `#[macro_export]` (modern idiom: exports to the crate root) or annotate the enclosing module with `#[macro_use]`.
- Multi-arm separator gotcha:
  - In standard `match` expressions, arms are separated by commas (`,`).
  - In `macro_rules!`, arms are separated by semicolons (`;`). Because TRPL only provides a single-arm example (`vec!`), this delimiter difference is an easy syntax trap.

23 - Conversions (Traits and Casting)

- Standard library conversion traits in `std::convert` and `std::str`:
  - `as` provides primitive casts (e.g. `len as f64`), truncating on overflow.
  - `From<T>` provides `Into<U>` automatically via a blanket implementation.
  - `FromStr` powers the generic `"str".parse::<T>()` method.
  - `TryFrom<T>` provides `TryInto<U>` automatically and returns `Result`.
  - `AsRef<T>` and `AsMut<T>` allow cheap reference-to-reference conversions.
  - Propagating inner errors with `?` can be unlocked either via `.map_err(...)` or by implementing `From<InnerError> for OuterError` (which `thiserror` automates via `#[from]`).

`as` Casting (`using_as.rs`)

```rust
fn average(values: &[f64]) -> f64 {
    let total = values.iter().sum::<f64>();
    total / values.len() as f64
}
```

`From` and `Into` (`from_into.rs`)

```rust
#[derive(Debug)]
struct Person {
    name: String,
    age: u8,
}

impl Default for Person {
    fn default() -> Self {
        Self {
            name: String::from("John"),
            age: 30,
        }
    }
}

impl From<&str> for Person {
    fn from(s: &str) -> Self {
        // Direct iterator unpack with let-else (zero allocations):
        let mut split = s.split(',');
        let (Some(name), Some(age), None) =
            (split.next(), split.next(), split.next())
        else {
            return Self::default();
        };

        /* Alternative: collect into Vec and slice-destructure:
        let parts: Vec<_> = s.split(',').collect();
        let &[name, age] = parts.as_slice() else {
            return Self::default();
        };
        */

        if name.is_empty() {
            return Self::default();
        }

        let Ok(age) = age.parse() else {
            return Self::default();
        };

        Self {
            name: name.into(),
            age,
        }
```

```
    }
}
```

`FromStr` (`from_str.rs`)

```rust
use std::num::ParseIntError;
use std::str::FromStr;

#[derive(Debug, PartialEq)]
struct Person {
    name: String,
    age: u8,
}

#[derive(Debug, PartialEq)]
enum ParsePersonError {
    BadLen,
    NoName,
    ParseInt(ParseIntError),
}

impl From<ParseIntError> for ParsePersonError {
    fn from(err: ParseIntError) -> Self {
        Self::ParseInt(err)
    }
}

/* With the thiserror crate, the above simplifies down to:
#[derive(thiserror::Error, Debug)]
enum ParsePersonError {
    #[error("Incorrect number of fields")]
    BadLen,

    #[error("Empty name field")]
    NoName,

    #[error("Invalid age")]
    ParseInt(#[from] ParseIntError), // auto-generates impl From<ParseIntError>
}
*/

impl FromStr for Person {
    type Err = ParsePersonError;

    fn from_str(s: &str) -> Result<Self, Self::Err> {
        let parts: Vec<_> = s.split(',').collect();

        let &[name, age] = parts.as_slice() else {
            return Err(ParsePersonError::BadLen);
```

```
        };

        if name.is_empty() {
            return Err(ParsePersonError::NoName);
        }

        // Uses the From<ParseIntError> implementation above:
        let age = age.parse()?;

        /* Alternative without implementing From:
        let age = age.parse().map_err(ParsePersonError::ParseInt)?;
        */

        Ok(Person {
            name: name.into(),
            age,
        })
    }
}
```

`TryFrom` and `TryInto` (`try_from_into.rs`)

```rust
use std::convert::{TryFrom, TryInto};

#[derive(Debug, PartialEq)]
struct Color {
    red: u8,
    green: u8,
    blue: u8,
}

#[derive(Debug, PartialEq)]
enum IntoColorError {
    BadLen,
    IntConversion,
}

/* We could shorten repetitive .map_err() calls by implementing From:
impl From<std::num::TryFromIntError> for IntoColorError {
    fn from(_: std::num::TryFromIntError) -> Self {
        Self::IntConversion
    }
}
// Then: let red: u8 = red.try_into()?;
*/

impl TryFrom<(i16, i16, i16)> for Color {
    type Error = IntoColorError;

    fn try_from(tuple: (i16, i16, i16)) -> Result<Self, Self::Error> {
        let (red, green, blue) = tuple;

        let red: u8 = red
            .try_into()
            .map_err(|_| IntoColorError::IntConversion)?;
        let green: u8 = green
            .try_into()
            .map_err(|_| IntoColorError::IntConversion)?;
        let blue: u8 = blue
            .try_into()
            .map_err(|_| IntoColorError::IntConversion)?;

        Ok(Self { red, green, blue })
    }
}
```

```rust
impl TryFrom<[i16; 3]> for Color {
    type Error = IntoColorError;

    fn try_from(arr: [i16; 3]) -> Result<Self, Self::Error> {
        let [red, green, blue] = arr;
        (red, green, blue).try_into()
    }
}

impl TryFrom<&[i16]> for Color {
    type Error = IntoColorError;

    fn try_from(slice: &[i16]) -> Result<Self, Self::Error> {
        let &[red, green, blue] = slice else {
            return Err(IntoColorError::BadLen);
        };

        (red, green, blue).try_into()
    }
}
```

`AsRef` and `AsMut` (`as_ref_mut.rs`)

```rust
// Obtain the number of bytes (not characters) in the given argument
// (`.len()` returns the number of bytes in a string).
fn byte_counter<T: AsRef<str>>(arg: T) -> usize {
    arg.as_ref().len()
}

// Obtain the number of characters (not bytes) in the given argument.
fn char_counter<T: AsRef<str>>(arg: T) -> usize {
    arg.as_ref().chars().count()
}

// Squares a number using `as_mut()`.
fn num_sq<T: AsMut<u32>>(arg: &mut T) {
    let n = arg.as_mut();
    *n *= *n;
}
```